

ภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เอกสารประกอบการสอน

040223101 การเขียนโปรแกรมคอมพิวเตอร์สำหรับคณิตศาสตร์ 1

(Computer Programming for Mathematics 1)

บทที่ 9

ฟังก์ชัน

(Functions)

ผู้สอน: รศ.ดร.อนุชิต จิตพัฒนกุล

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อศึกษาบทนี้จบ นักศึกษาสามารถ

- อธิบายประโยชน์ของฟังก์ชันในการเขียนโปรแกรมแบบมอดูลได้
- นิยามฟังก์ชันด้วยคำสั่ง def พร้อมพารามิเตอร์และการคืนค่าด้วย return ได้
- แยกความแตกต่างระหว่างพารามิเตอร์กับอาร์กิวเมนต์ และเข้าใจขอบเขตตัวแปร (scope) ได้
- ใช้ค่าปริยาย (default value) และการคืนค่าหลายค่าได้
- ประยุกต์ฟังก์ชันและแนวคิดการเรียกซ้ำ (recursion) เบื้องต้นแก้ปัญหาทางคณิตศาสตร์ได้

CLO: เนื้อหาบทนี้สอดคล้องกับผลลัพธ์การเรียนรู้ของรายวิชา (CLO 4)

บทที่ 9 ฟังก์ชัน

เมื่อโปรแกรมใหญ่ขึ้น โค้ดบางส่วนมักถูกใช้ซ้ำหลายครั้ง การคัดลอกโค้ดเดิมไปวางหลาย ๆ ทำให้แก้ไข ยากและเกิดข้อผิดพลาดง่าย **ฟังก์ชัน (function)** ช่วยให้เรารวมกลุ่มคำสั่งไว้ในชื่อเดียว แล้วเรียกใช้ ซ้ำได้ทุกที่ ทำให้โปรแกรมเป็นระเบียบ อ่านง่าย และบำรุงรักษาสะดวก บทนี้จะศึกษาการสร้างและใช้งาน ฟังก์ชัน

9.1 การนิยามและเรียกใช้ฟังก์ชัน

นิยามฟังก์ชันด้วยคำสั่ง **def** ตามด้วยชื่อฟังก์ชันและวงเล็บ ภายในเก็บบล็อกคำสั่งที่เยื้องเข้าไป เมื่อต้องการใช้งานจึง "เรียก" ฟังก์ชันด้วยชื่อตามด้วยวงเล็บ

ไฟล์ 09func1.py

```
def greet():  
    print("สวัสดีครับ")  
    print("ยินดีตอนรับ")  
  
greet()      # เรียกใช้ครั้งที่ 1  
greet()      # เรียกใช้ครั้งที่ 2
```

ผลลัพธ์:

```
สวัสดีครับ  
ยินดีตอนรับ  
สวัสดีครับ  
ยินดีตอนรับ
```

9.2 พารามิเตอร์และอาร์กิวเมนต์

ฟังก์ชันรับข้อมูลเข้ามาทำงานผ่าน **พารามิเตอร์ (parameter)** ที่ระบุในวงเล็บตอนนิยาม ส่วนค่าจริงที่ ส่งเข้าไปตอนเรียกใช้ เรียกว่า **อาร์กิวเมนต์ (argument)**

ไฟล์ 09param.py

```
def greet(name):  
    print(f"สวัสดีคุณ {name}")  
  
greet("Somchai")  
greet("Suda")
```

ผลลัพธ์:

```
สวัสดีคุณ Somchai  
สวัสดีคุณ Suda
```

9.3 การคืนค่าด้วย return

คำสั่ง `return` ใช้ส่งผลลัพธ์กลับออกจากฟังก์ชันเพื่อนำไปใช้ต่อ เมื่อพบ `return` ฟังก์ชันจะจบการทำงานทันที ฟังก์ชันที่คืนค่าต่างจากฟังก์ชันที่เพียงแสดงผลด้วย `print` ตรงที่ค่าที่คืนสามารถนำไปคำนวณ ต่อได้

ไฟล์ 09return.py

```
def square(x):  
    return x * x  
  
result = square(5)  
print(result)  
print(square(3) + square(4))
```

ผลลัพธ์:

```
25  
25
```

ตัวอย่างฟังก์ชันคำนวณพื้นที่สามเหลี่ยมที่รับพารามิเตอร์สองตัว

ไฟล์ 09area.py

```
def triangle_area(base, height):  
    return 0.5 * base * height  
  
print(triangle_area(10, 6))  
print(triangle_area(7, 4))
```

ผลลัพธ์:

```
30.0  
14.0
```

9.4 ค่าปริยาย (Default Value)

เราสามารถกำหนดค่าปริยายให้พารามิเตอร์ได้ หากเรียกใช้โดยไม่ส่งอาร์กิวเมนต์ตัวนั้น ฟังก์ชันจะใช้ค่า ปริยายแทน

ไฟล์ 09default.py

```
def power(base, exp=2):  
    return base ** exp  
  
print(power(5))          # ใช้ exp=2  
print(power(5, 3))       # กำหนด exp=3
```

ผลลัพธ์:

```
25  
125
```

9.5 การคืนค่าหลายค่า

ฟังก์ชันใน Python คืนค่าหลายค่าพร้อมกันได้โดยคั่นด้วยจุลภาค ผู้เรียกใช้สามารถรับค่าหลายตัวแปรได้

ไฟล์ 09multi.py

```
def stat(numbers):  
    return min(numbers), max(numbers), sum(numbers) / len(numbers)  
  
low, high, avg = stat([4, 8, 15, 16, 23, 42])  
print(f"ต่ำสุด={low}, สูงสุด={high}, เฉลี่ย={avg:.2f}")
```

ผลลัพธ์:

ต่ำสุด=4, สูงสุด=42, เฉลี่ย=18.00

9.6 ขอบเขตของตัวแปร (Scope)

ตัวแปรที่ประกาศภายในฟังก์ชันเป็น **ตัวแปรเฉพาะที่ (local variable)** ใช้ได้เฉพาะภายในฟังก์ชันนั้น เมื่อฟังก์ชันจบการทำงานตัวแปรจะถูกลบ ส่วนตัวแปรที่ประกาศนอกฟังก์ชันเป็น **ตัวแปรส่วนกลาง (global variable)** การแยกขอบเขตเช่นนี้ช่วยป้องกันการรบกวนกันระหว่างฟังก์ชัน

ไฟล์ 09scope.py

```
def f():  
    y = 10          # ตัวแปรเฉพาะที่  
    print("ในฟังก์ชัน y =", y)  
  
f()  
# print(y)         # จะเกิด NameError เพราะ y อยู่เฉพาะในฟังก์ชัน
```

ผลลัพธ์:

ในฟังก์ชัน y = 10

9.7 การเรียกซ้ำ (Recursion) เบื้องต้น

ฟังก์ชันสามารถเรียกตัวเองได้ เรียกว่า การเรียกซ้ำ (recursion) เหมาะกับปัญหาที่นิยามด้วยตัวเอง เช่น แฟกทอเรียล $n! = n \times (n-1)!$ สิ่งสำคัญคือต้องมี **กรณีฐาน (base case)** เพื่อหยุดการเรียกซ้ำ

ไฟล์ 09recursion.py

```
def factorial(n):
    if n == 0 or n == 1:    # กรณีฐาน
        return 1
    return n * factorial(n - 1)

print(factorial(5))
print(factorial(6))
```

ผลลัพธ์:

120
720

ข้อควรระวัง: หากลืมกรณีฐานหรือเงื่อนไขไม่ลดลงสู่กรณีฐาน การเรียกซ้ำจะไม่มีวันจบและเกิดข้อผิดพลาด RecursionError

9.8 สรุปท้ายบท

ฟังก์ชันช่วยรวมกลุ่มคำสั่งให้เรียกใช้ซ้ำได้ ทำให้โปรแกรมเป็นมอดูลและบำรุงรักษาง่าย นิยามด้วย def รับข้อมูลผ่านพารามิเตอร์ และส่งผลลัพธ์กลับด้วย return เราสามารถกำหนดค่าปรีาย คำนวณค่าหลายค่า และ ต้องเข้าใจขอบเขตตัวแปรเฉพาะที่กับ ส่วนกลาง การเรียกซ้ำเป็นเทคนิคที่ฟังก์ชันเรียกตัวเองโดยต้องมีกรณี ฐานเสมอ

แบบฝึกหัดท้ายบท

- จงเขียนฟังก์ชัน is_even(n) ที่คืนค่า True ถ้า n เป็นเลขคู่ และ False ถ้าเป็นเลขคี่
- จงเขียนฟังก์ชัน circle(r) ที่คืนทั้งเส้นรอบวงและพื้นที่ของวงกลมรัศมี r
- จงเขียนฟังก์ชัน maximum(a, b, c) ที่คืนค่าที่มากที่สุดในสามจำนวน
- จงเขียนฟังก์ชัน sum_to(n) ที่คืนผลรวม $1 + 2 + \dots + n$ โดยใช้การเรียกซ้ำ
- จงเขียนฟังก์ชัน fibonacci(n) ที่คืนพจน์ที่ n ของลำดับฟีโบนัชชี

เฉลยแบบฝึกหัดบทที่ 9

ข้อ 1

```
def is_even(n):  
    return n % 2 == 0  
  
print(is_even(8), is_even(7))
```

ผลลัพธ์:

True False

ข้อ 2

```
def circle(r):  
    pi = 3.14159  
    return 2 * pi * r, pi * r ** 2  
  
c, a = circle(5)  
print(f"เส้นรอบวง={c:.2f}, พื้นที่={a:.2f}")
```

ผลลัพธ์:

เส้นรอบวง=31.42, พื้นที่=78.54

ข้อ 4 ผลรวมแบบเรียกซ้ำ

```
def sum_to(n):  
    if n == 1:  
        return 1  
    return n + sum_to(n - 1)  
  
print(sum_to(100))
```

ผลลัพธ์:

5050

ข้อ 5 ฟิโบนัชชี

```
def fibonacci(n):  
    if n <= 2:  
        return 1  
    return fibonacci(n - 1) + fibonacci(n - 2)  
  
for i in range(1, 9):  
    print(fibonacci(i), end=" ")
```

ผลลัพธ์:

1 1 2 3 5 8 13 21

